

Chaitin–Gödel I in Basic Recursive Arithmetic: an object-level diagonal program and an internal implication

1 Chaitin’s argument, recalled

Chaitin’s proof of Gödel’s first incompleteness theorem [1, 2] runs as follows. Fix a sound theory T that interprets enough arithmetic, and fix a universal partial computable interpreter U with a Kolmogorov-complexity bound $K_U(x) := \min\{|p| : U(p) \downarrow = x\}$. There is a constant c (depending only on T and U) such that, for every L^* and every integer x' ,

$$T \vdash K_U(\underline{x'}) > L^* \implies T \text{ is inconsistent, provided } L^* > c + \log L^*.$$

The contradiction is exhibited by a single short program g_{L^*} : enumerate $\text{thmT}(0), \text{thmT}(1), \dots$ until a proof of the form $\text{code}K_U(?) > L^*$ is found, then output the witnessed integer x' . The size of g_{L^*} is $c + \log L^*$, strictly below L^* , so the very witness x' that T proves to be K_U -incompressible is itself g_{L^*} -output — a contradiction to the size lemma internal to T .

2 The Basic Recursive Arithmetic statement

The formalisation packages Chaitin’s argument as a single closed T -derivation: it produces, from any putative T -proof that $\text{thmT } w$ asserts incompressibility of its own read-back, a T -proof that a constructed code is a proof of $0 = 1$. Two design choices make the statement sharp.

1. The “construct the short program g_{L^*} ” step is a genuine Basic Recursive Arithmetic combinator $\mathbf{G} : \text{Fun}_1$, an honest object-level program, not a meta-level term transformer. The constructed proof code is literally $\text{ap}_1 \mathbf{G} w$.
2. The “if $\dots$ then inconsistent” step is an *internal* Hilbert implication $\Rightarrow$ inside a single Deriv , not a meta-level function $\text{Deriv } X \rightarrow \text{Deriv } Y$. The deduction content lives inside the object logic.

The headline theorem (`BRA4/CgFunFun1.agda`, `chaitinG1_Fun1.all`) is:

$$\begin{aligned} (w : \text{Term}) \rightarrow \text{Deriv} \left(\underbrace{\text{eqF} (\text{ap}_1 \text{thmT } w) (\text{ap}_1 \text{Kcode}_{L^*} (\text{ap}_1 \text{outKdef}_{L^*} w))}_{\text{firing condition at } w} \right. \\ \left. \Rightarrow \underbrace{\text{eqF} (\text{ap}_1 \text{thmT} (\text{ap}_1 \mathbf{G} w)) \text{codeFalse}}_{\text{ap}_1 \mathbf{G} w \text{ proves } 0=1} \right). \end{aligned}$$

Here w is an *arbitrary* Term: no closure hypothesis, no `isNat` integrality predicate, no `Con(T)` premise, and no closedness bookkeeping. In words: *for every* w , T proves “if $\text{thmT } w$ is the

incompressibility statement Kcode_{L^*} about its own read-back $\text{outKdef}_{L^*} w$, then the code $\text{ap}_1 G w$ is a T -proof of $0 = 1$.

The antecedent is the firing condition in **num**-raw form: the subject slot is $\text{num } x'$ for an arbitrary Term x' , with no integrality required to align with the read-back. Whether $\text{thmT } w$ has the form $\text{Kcode}_{L^*} (-)$ is read off internally by the Basic Recursive Arithmetic function $\text{outKdef}_{L^*} = \text{decode} \circ \text{projKdef} \circ \text{thmT}$, so the witnessed integer is forced to be $\text{outKdef}_{L^*} w$.

The program G is obtained by bracket abstraction: the Chaitin construction is a fixed one-variable object-level combinator context, and $G : \text{Fun}_1$ is its abstraction, with $\text{ap}_1 G w$ equal to that construction — as a *proved* T -derivation, via combinatory completeness, not a definitional identity (`BRA4/AbsFun1.agda`, `BRA4/CgFunFun1.agda`). The implication is the deduction-theorem internalisation of the firing step (`BRA4/CgFalseImp.agda`). Because G is an honest combinator, the statement holds at an arbitrary w with no closedness side conditions.

3 The subtle part: the internal evaluator and majorisation

The conclusion ultimately rests on an *object-level* run of the universal interpreter evalU on the looping program g_{L^*} embedded in G . This is where the formalisation is delicate, and it is the part worth documenting carefully. Indeed, the main technical novelty of the formalisation is not the diagonal argument itself — a faithful rendering of Chaitin’s — but the construction of *object-level fuel majorants* that let evaluator completeness be expressed and proved entirely inside Basic Recursive Arithmetic. The diagonalisation is classical; the Term-valued fuel of §3.2 is not.

3.1 Representation of the universal interpreter

Inside the formalisation, evalU is realised as a small CK-machine: configurations carry a control Term together with a continuation stack, and a one-step transition function stepU advances one rule at a time. Running the interpreter for fuel n is iteration of stepU from a starting configuration. A second layer wraps the μ -search loop of g_{L^*} , so that the interpreter’s outer behaviour matches Chaitin’s informal description “enumerate $\text{thmT } 0, \text{thmT } 1, \dots$ until a fitting proof is found”. The rule-by-rule layout of stepU is not what makes the proof delicate; what matters is the *shape* of the completeness statement the proof needs.

3.2 Gandy–Howard majorisation for Term-fuel completeness

A first attempt at a completeness statement would say: for every *numeral* fuel $\bar{n}$ large enough, iterating stepU on g_{L^*} at $\bar{n}$ reaches the final configuration $\text{cfgRT}(\text{num } x') K_0$. Such a statement is useless inside an internal **Deriv**-argument: thmT , evalU , and stepU all take Terms, not meta-Nats, so the fuel slot of the resulting thmT -fact must itself be a Term. The obstruction is precise: since Theorem 13 internalises only statements about Basic Recursive Arithmetic terms, a completeness theorem quantified over meta-level naturals cannot be fed into the later derivation. The completeness theorem actually needed has the form

$$(x : \text{Term}) \rightarrow \text{Deriv}(\text{iter stepU}(\text{cfgEV}(\text{mcode } f) x K_0) (\text{fuelG } f x) = \text{cfgRT}(\text{num}(f x)) K_0),$$

i.e. Term-input/Term-fuel completeness, where $\text{fuelG } f$ is a closed Basic Recursive Arithmetic combinator. Proving such a statement by structural induction on the combinator tree of f stalls at the R -recursion node: the inner induction would need numerical control over how much fuel the recursive calls consume, but the actual running time is not available at the **Deriv**-level.

The resolution adapts the classical Gandy–Howard majorisation method [3, 4]. One builds, by primitive recursion on the structure of f , a closed Basic Recursive Arithmetic combinator $\text{fuelG } f : \text{Fun}_1$ that provably *majorises* the numerical fuel any actual stepU -run consumes at every Term input. The induction-with-substituted-motive then carries the stable shape “the run completes at *any* $\text{fuel} \geq \text{fuelG } f \ x$ ”, which descends through the R -node because fuelG is itself defined by a recursive scheme that mirrors f . The μ -loop wrapping g_{L^*} requires the same idea one level higher: a majorising combinator fuelMu controls the outer search depth and is layered on top of fuelG by the fuel-composition lemma iter_add_T . The closed Term fuel that finally appears inside $\text{ap}_1 \text{G } w$ is

$$n_T := \text{fuelMu } k_{\max} k_{\max} + \text{fuelG } k_{\max} O, \quad k_{\max} = \text{gRec } O (s (\text{closeW } w)),$$

and the resulting positive thmT -fact

$$\text{thmT cPos} \stackrel{\text{Deriv}}{=} \text{coderunProg } g_L n_T = s x'$$

follows from this run via Theorem 13 (singular). Without majorisation there would simply be no Term to put in the fuel slot.

3.3 Closing the chain

With Term-fuel completeness in hand, the operational core closes the chain

$$\text{evalU} (\text{parse gLnameDef}) n_T = s (\text{outKdef}_{L^*} k_{\max})$$

through seven internal segments (the module ChainKdef), glued by iter_add_T . The positive thmT -fact above is its internalisation via Theorem 13. The negative leg $\text{thmT} (\text{cmp outerWrap cSize}) = \text{cNeg coderunProg } g_L n_T = s x'$ is obtained from the firing condition by two thmT_at_sb substitutions and an encoded_mp against the size atom. The clash is then a single $\text{encoded_mp} / \text{encoded_exfalse}$ pair, internal throughout under the $\Rightarrow$ -context.

3.4 No consistency hypothesis

Chaitin’s original argument concludes “ T is inconsistent” (in the metatheory). The formalisation strengthens this: the conclusion is an *internal* implication, a single Basic Recursive Arithmetic derivation that, on the firing condition, the constructed code $\text{ap}_1 \text{G } w$ is a T -proof of $0 = 1$. No $\text{Con}(T)$ premise is consumed, no isNat integrality predicate is consumed, and the firing condition is stated in num-rw form.

This is the precise interface offered to the later surprise-Gödel-II descent: that descent will *not* need to reprove any evaluator completeness, and will use the implication purely as a black-box arrow from a Deriv -witness of the firing condition to a Deriv -witness of codeFalse . The Term-fuel completeness work paid for here — and in particular the Gandy–Howard machinery of §3.2 — is invisible to the descent. The contrast with consistency is sharp: the implication itself consumes no $\text{Con}(T)$, but the surprise-Gödel-II argument will consume internal consistency *after*, and only after, the implication has been discharged.

Remark 1 (Lineage). *Although the result formalises Chaitin’s theorem, its shape — an object-level proof predicate, a constructed code, and an internal implication $A \rightarrow \text{Proof}(p, 0 = 1)$ — places it closer to the formal-provability tradition of Hilbert–Bernays [5], Löb [6], and Feferman [7], and to its modern mechanisations [8], than to Chaitin’s original metatheoretic presentation. What is proved here is not the metatheorem “ $T \vdash A$ implies T inconsistent” but the single internal derivation of the corresponding implication, in the style of those derivability-condition analyses.*

4 Files

- `BRA4/AbsFun1.agda` — the bracket-abstraction DSL in one variable, and the combinatory-completeness evaluation (a proved `Deriv`).
- `BRA4/CgFunFun1.agda` — the genuine Fun_1 diagonal `G` and the headline `chaitinG1.Fun1_all`.
- `BRA4/CgFalseImp.agda` — the internal implication (deduction-theorem internalisation) and its fresh-witness instance.
- `BRA4/CloseW.agda` — `closeW` and the structural lemmas it supports.

All files typecheck under `--safe --without-K --exact-split` with no holes and no postulates.

References

- [1] G. J. Chaitin, “Computational complexity and Gödel’s incompleteness theorem,” *ACM SIGACT News*, no. 9, pp. 11–12, 1971.
- [2] G. J. Chaitin, “Information-theoretic limitations of formal systems,” *Journal of the ACM*, vol. 21, no. 3, pp. 403–424, 1974.
- [3] W. A. Howard, “Hereditarily majorizable functionals of finite type,” in A. S. Troelstra (ed.), *Metamathematical Investigations of Intuitionistic Arithmetic and Analysis*, Lecture Notes in Mathematics, vol. 344, Springer, 1973, pp. 454–461.
- [4] R. O. Gandy, “Proofs of strong normalization,” in J. P. Seldin and J. R. Hindley (eds.), *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, Academic Press, 1980, pp. 457–477.
- [5] D. Hilbert and P. Bernays, *Grundlagen der Mathematik*, vol. II, Springer, 1939.
- [6] M. H. Löb, “Solution of a problem of Leon Henkin,” *Journal of Symbolic Logic*, vol. 20, no. 2, pp. 115–118, 1955.
- [7] S. Feferman, “Arithmetization of metamathematics in a general setting,” *Fundamenta Mathematicae*, vol. 49, pp. 35–92, 1960.
- [8] L. C. Paulson, “A mechanised proof of Gödel’s incompleteness theorems using Nominal Isabelle,” *Journal of Automated Reasoning*, vol. 55, no. 1, pp. 1–37, 2015.